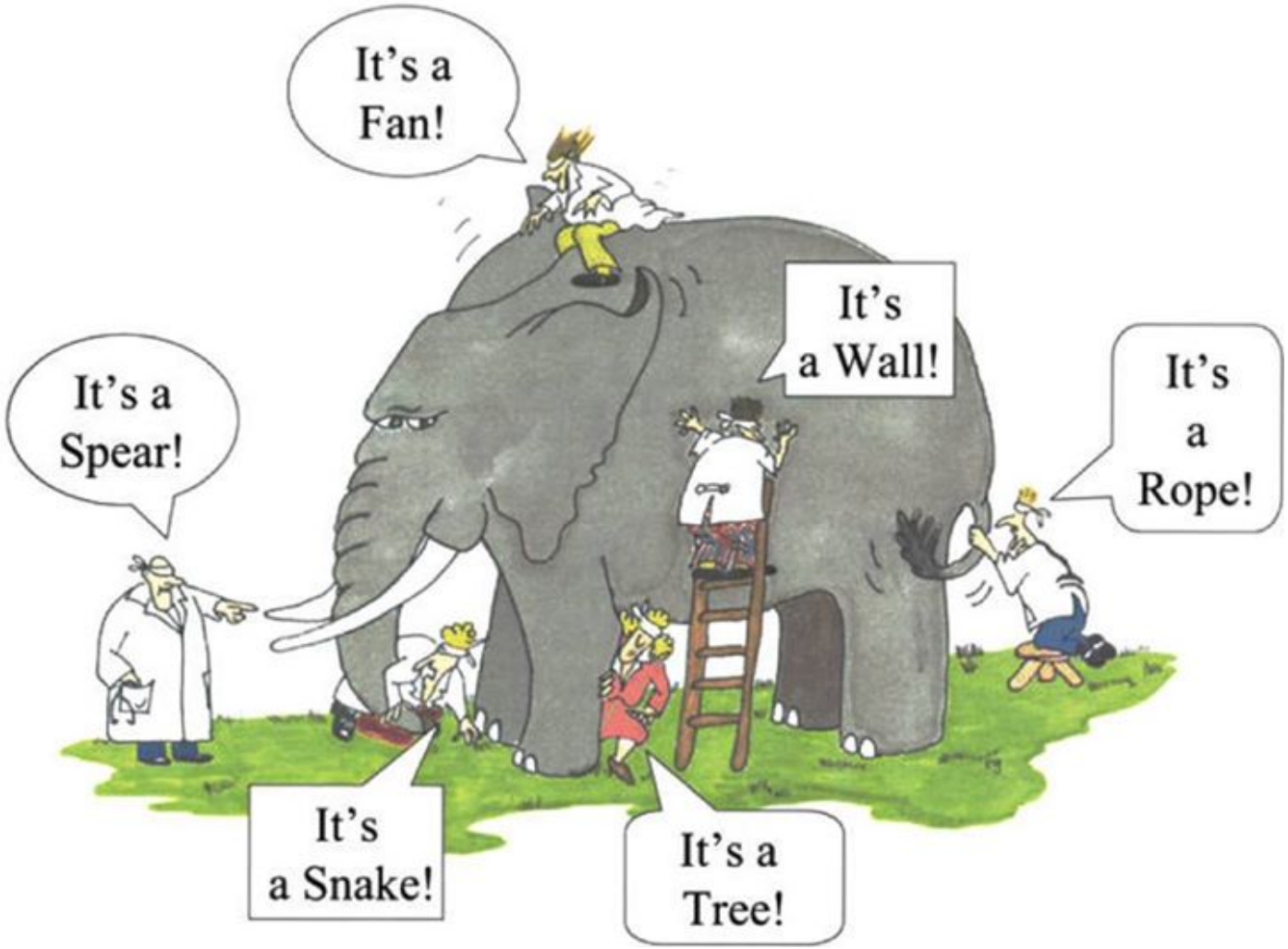
Outlook of the unit

4 **Ensemble methods**

- Bagging
- Random forests
- Boosting

Idea of ensemble methods



Blind men and an elephant

Bagging

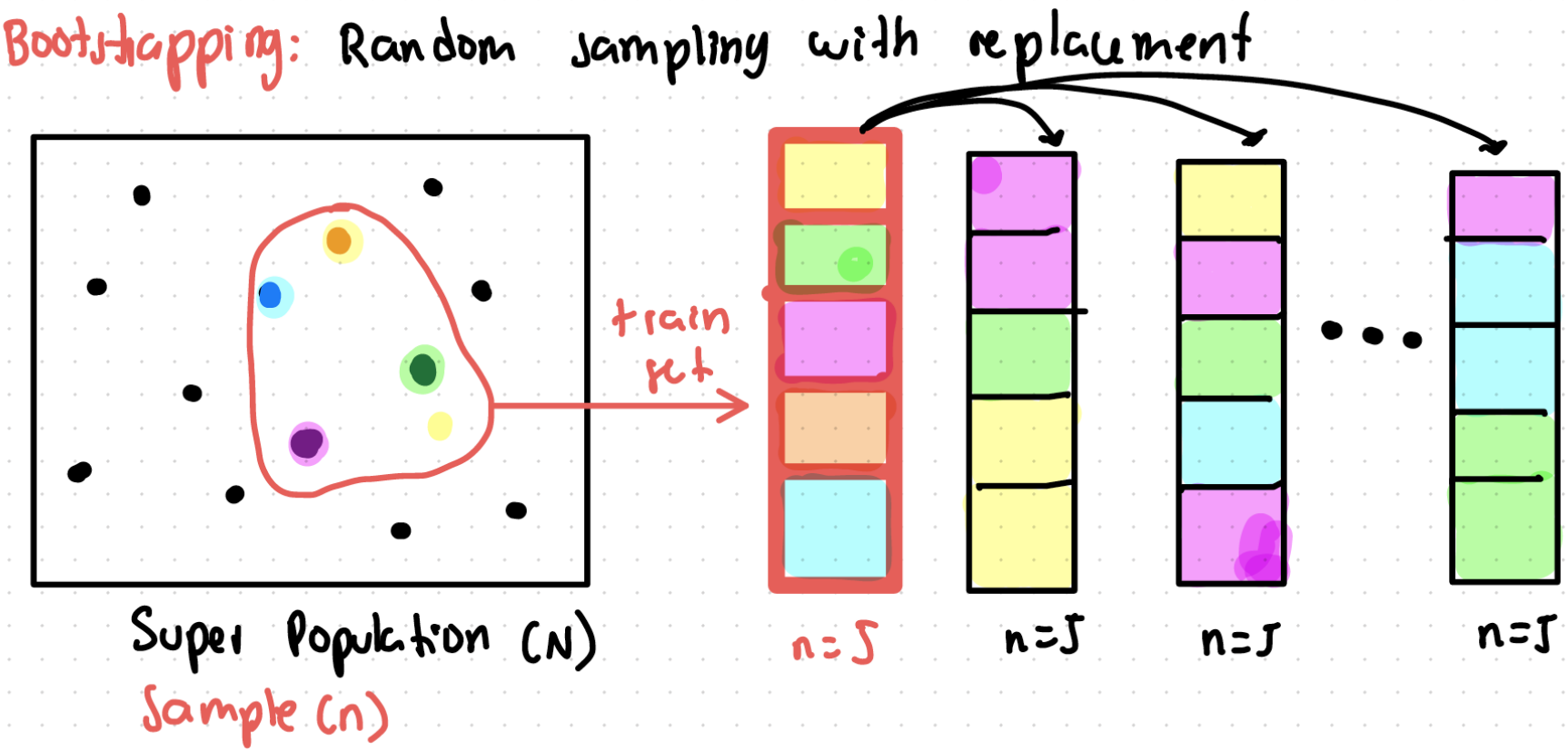
Bagging: Introduction

- The decision trees suffer from high variance.
- If we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.
- A procedure with low variance will yield similar results if applied repeatedly to distinct data sets (like linear regression).
- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method.
- Recall that given a set of n independent observations $Z_1, ..., Z_n$ each with variance σ^2, the variance of $\bar{Z}$ is given by σ^2/n.
- In other words, averaging a set of observations reduces variance.

Bagging = Bootstrap and aggregation

- A way to reduce the variance (increase prediction accuracy) of a method is to take many training sets, build prediction models using each training set, and average the predictions.
- This is not practical because we generally do not have access to multiple training sets.
- Instead, we can bootstrap, by taking repeated samples from the (single) training data set.

Bootstrapping idea



Bagging procedure for regression trees

1. Generate B different bootstrapped training data sets.
2. Train the method on the b-th bootstrapped training set in order to get $\hat{f}^{*,b}(x)$ for $b = 1, ..., B$ (for instance, construct regression trees without pruning).
3. Average all the predictions to obtain

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^{*,b}(x).$$

Bagging in the context of classification trees

We have described the bagging procedure in the regression context, to predict a quantitative outcome Y.

Bagging for classification trees:

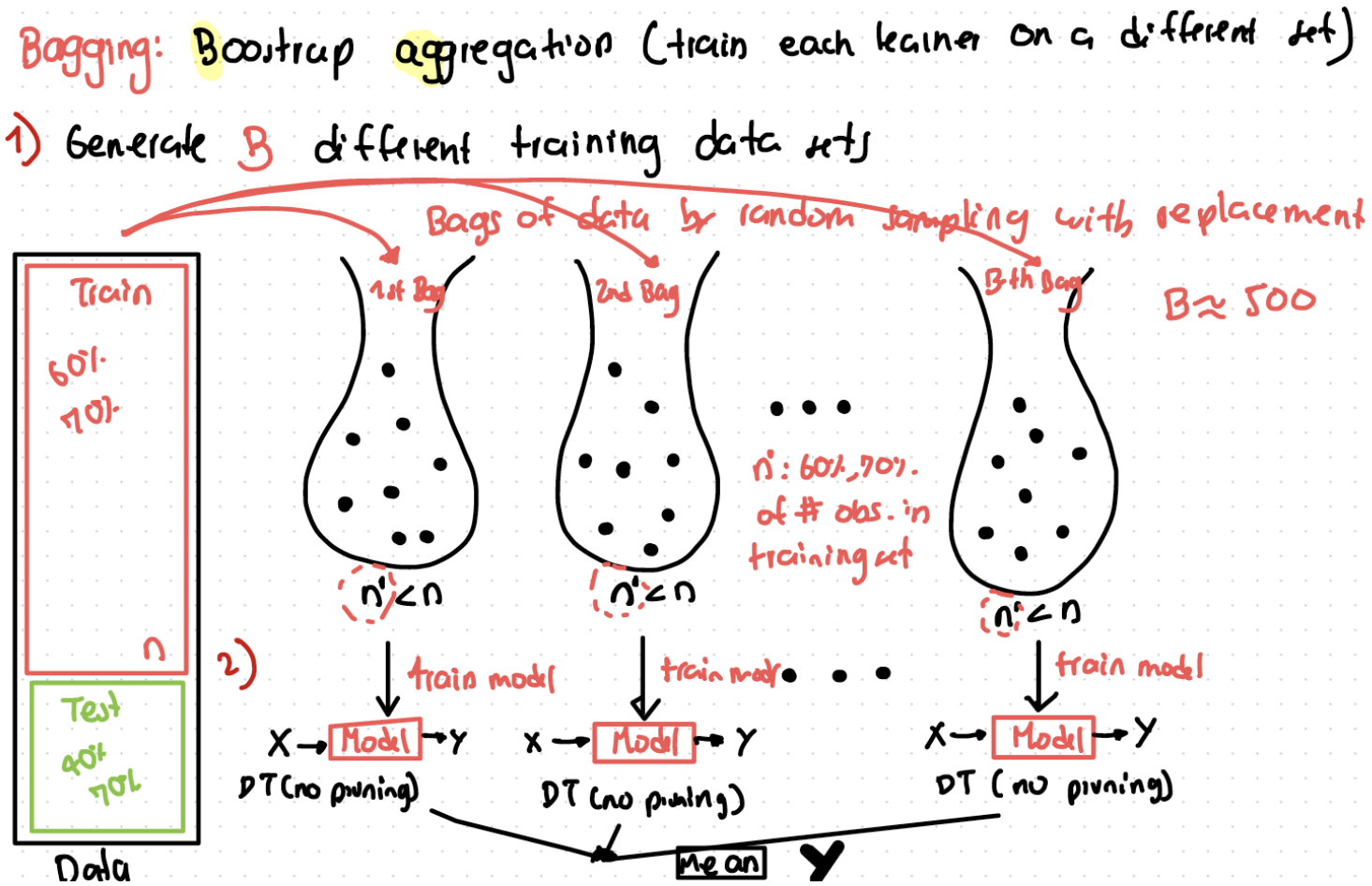
1. Generate B different bootstrapped training data sets.
2. Train the method on the b-th bootstrapped training set in order to record the class predicted by each of the B trees for a observation.
3. Take a majority vote: the overall prediction is the most commonly occurring majority class among the B predictions.

Comment:
Using a very large value of B will not lead to overfitting. In practice we use a value of B sufficiently large that the error has settled down, for instance $B = 500$.

Variable importance measure

- When we bag a large number of trees, it is not possible to represent the resulting statistical learning procedure using a single tree.
- Thus, bagging improves prediction accuracy at the expense of interpretability.
- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.

Bagging idea



Bagging regression trees in R: Boston housing data

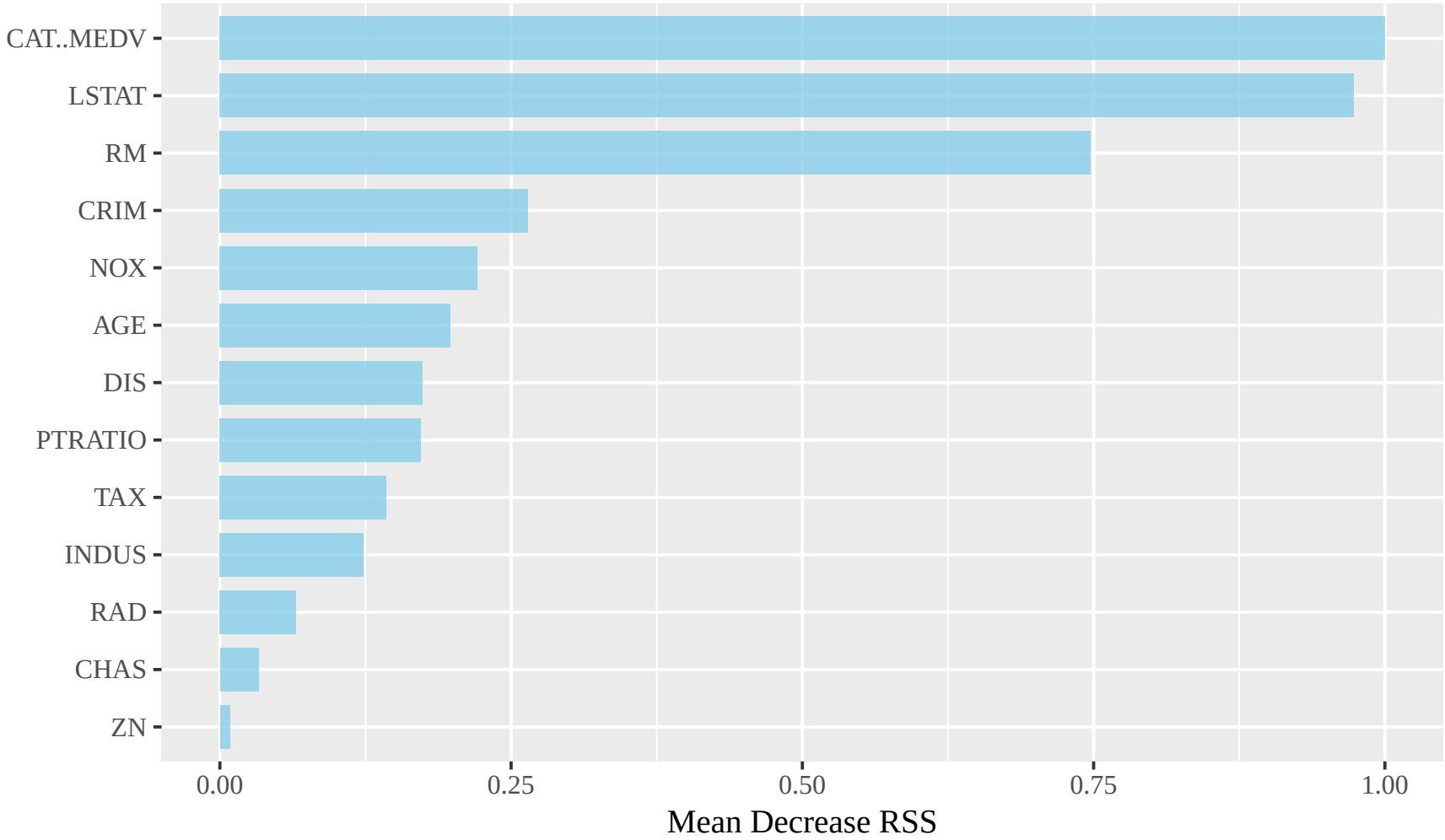
The `randomForest()`-command constructs a bagged tree and returns the results as an object.

```
# Loading libraries and the data
library(randomForest)

# Bagging a regression tree with all variables
> fit<-randomForest(MEDV~.,data=BostonH,mtry=13,
    importance =TRUE,subset=inTrain)
> fit
Call:
randomForest(formula = MEDV ~ ., data = BostonH, mtry =
    13, importance = TRUE, subset = inTrain)
   Type of random forest: regression
   Number of trees: 500
   No. of variables tried at each split: 13
```

- `mtry` indicates that all 13 predictors should be considered for each split of the tree.

Bagging regression trees in R: Variable importance measure



Random forests

Bagging to random forests

- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- Consequently, all of the bagged trees will look quite similar to each other.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.
- Random forests provide an improvement over bagged trees
- It offers a solution to correlated bagged trees: allows a random sample of m out of p predictors at each split in a tree in order to expose different features to be explored by an algorithm.

Random forests

- We build a number of decision trees on bootstrapped training samples.
- When building these decision trees, each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors.
- The split is allowed to use only one of those m predictors.
- A fresh selection of m predictors is taken at each split, and typically we choose $m \approx \sqrt{p}$ or by cross-validation.
- If a random forest is built using $m = p$, then this amounts simply to bagging.

Random forests in R: Boston housing data

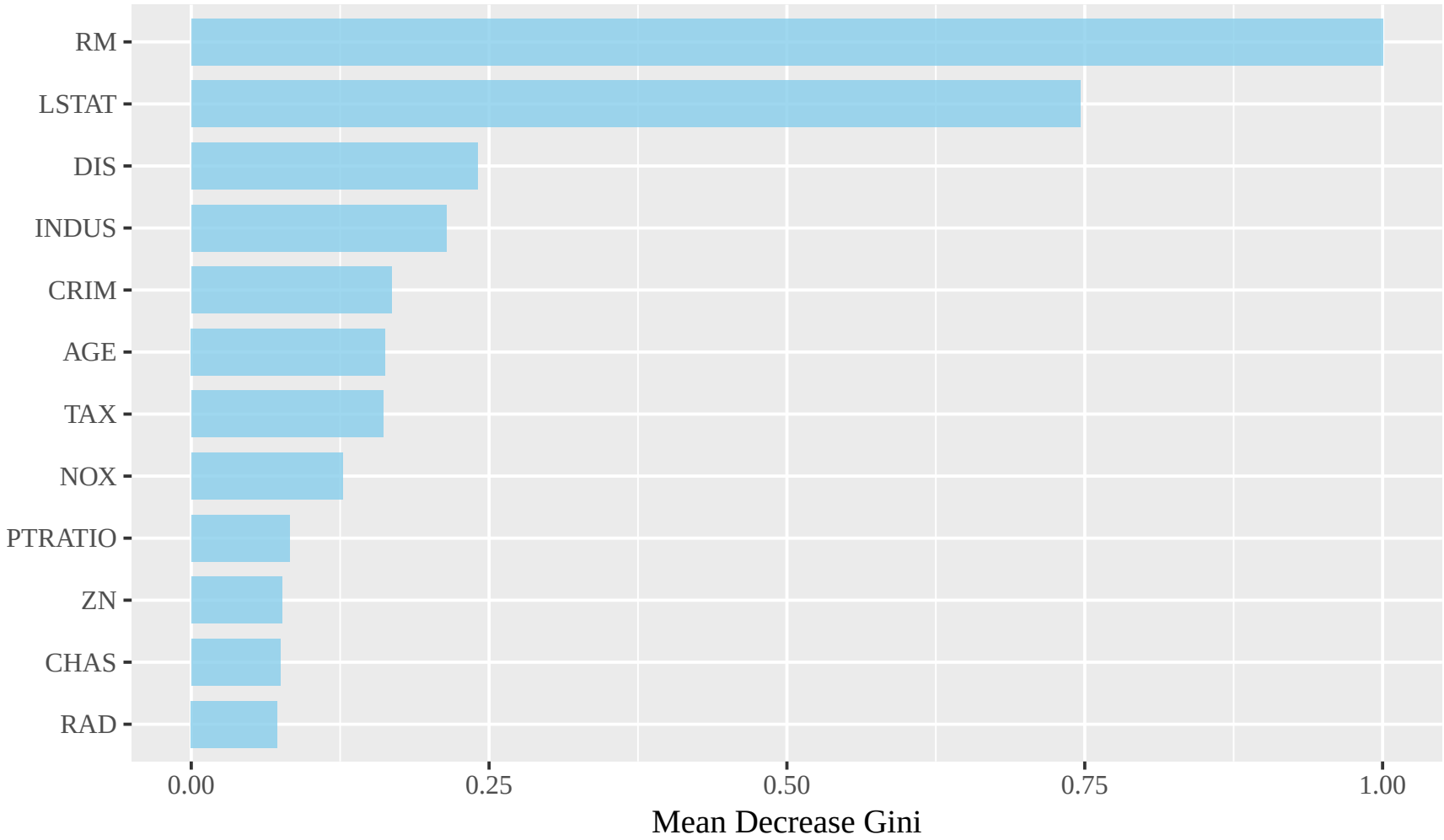
The `randomForest()`-command constructs random forests and returns the results as an object.

```
# Bagging a regression tree with all variables
> fit<-randomForest(MEDV_Fac~.,data=BostonH,mtry=sqrt(12)
    , importance =TRUE,subset=inTrain)
> fit

   Type of random forest: classification
   Number of trees: 500
   No. of variables tried at each split: 3
   OOB estimate of  error rate: 3.93%

Confusion matrix:
      Below Above class.error
Below   289     5   0.0170068
Above     9    53   0.1451613
```

Random forests in R: Variable importance measure



Pros and cons of random forest

Pros

- high predictive performance
- low chance of overfitting
- robustness
- easy to parallelize

Cons

- relatively long training time
- lack of interpretability (possible solution: variable importance measures)
- cannot incorporate missing values as easily as simple trees

Boosting

Boosting

Boosting is another technique for improving predictive accuracy of decision trees. It 'boosts' the performance of weak (only marginally better than random) classifiers.

Unlike bagging, in boosting trees are grown sequentially: a tree uses information from previously grown trees. Final classifier is built by combining sequentially updated classifiers.

Some popular boosting approaches:

- AdaBoost
- Stochastic Gradient Boosting

AdaBoost

Process:

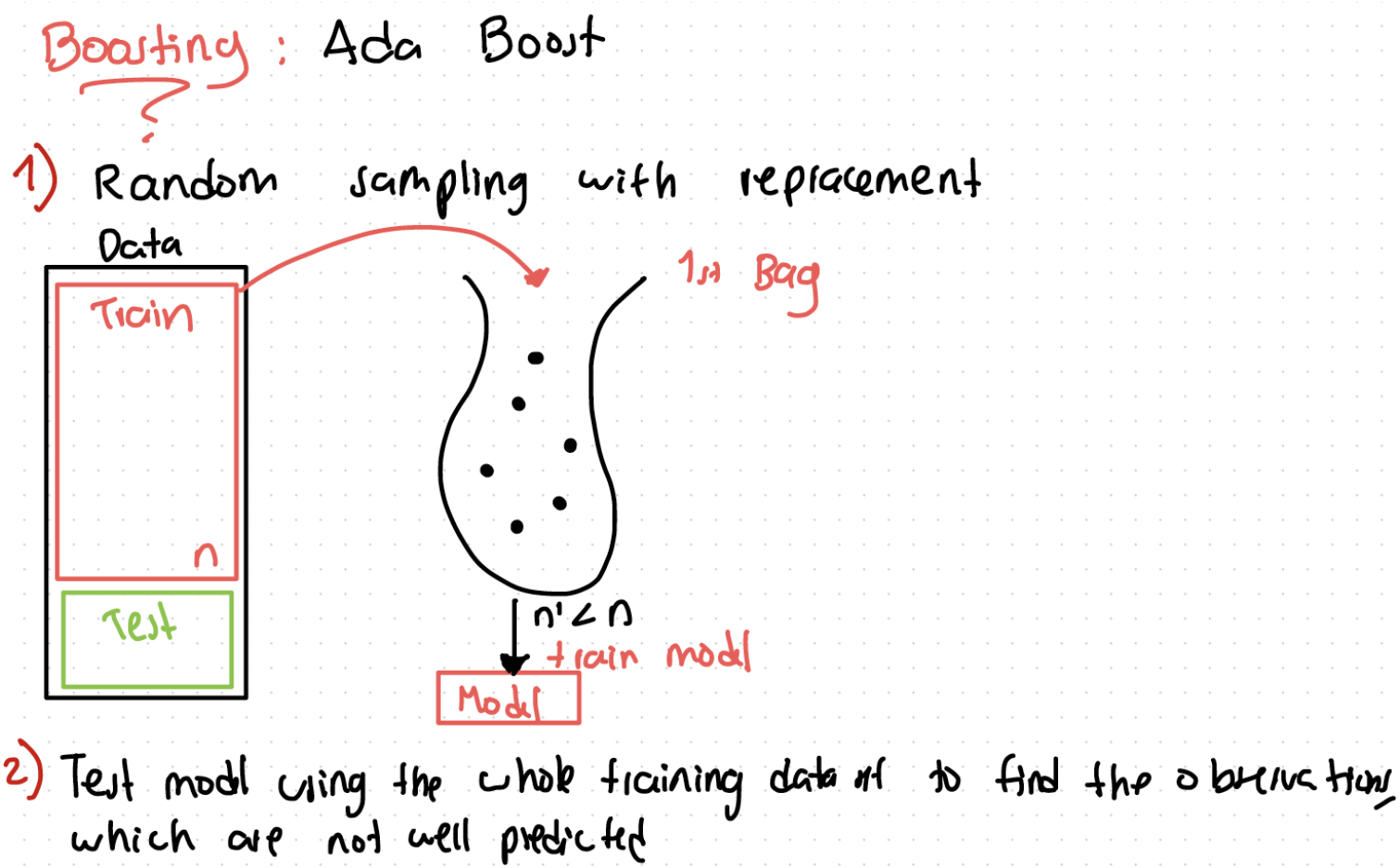
1. Initially each observation has the same starting weight $(1/n)$, an initial 'base' classifier is built and misclassification error is calculated.
2. Weights are updated in such a way that incorrectly predicted observations receive more weight and correctly classified less. A new classifier with updated weights is built and this step is repeated M times.
3. Final classifier is calculated as a weighted sum of M base classifiers.

Tuning parameters for boosting

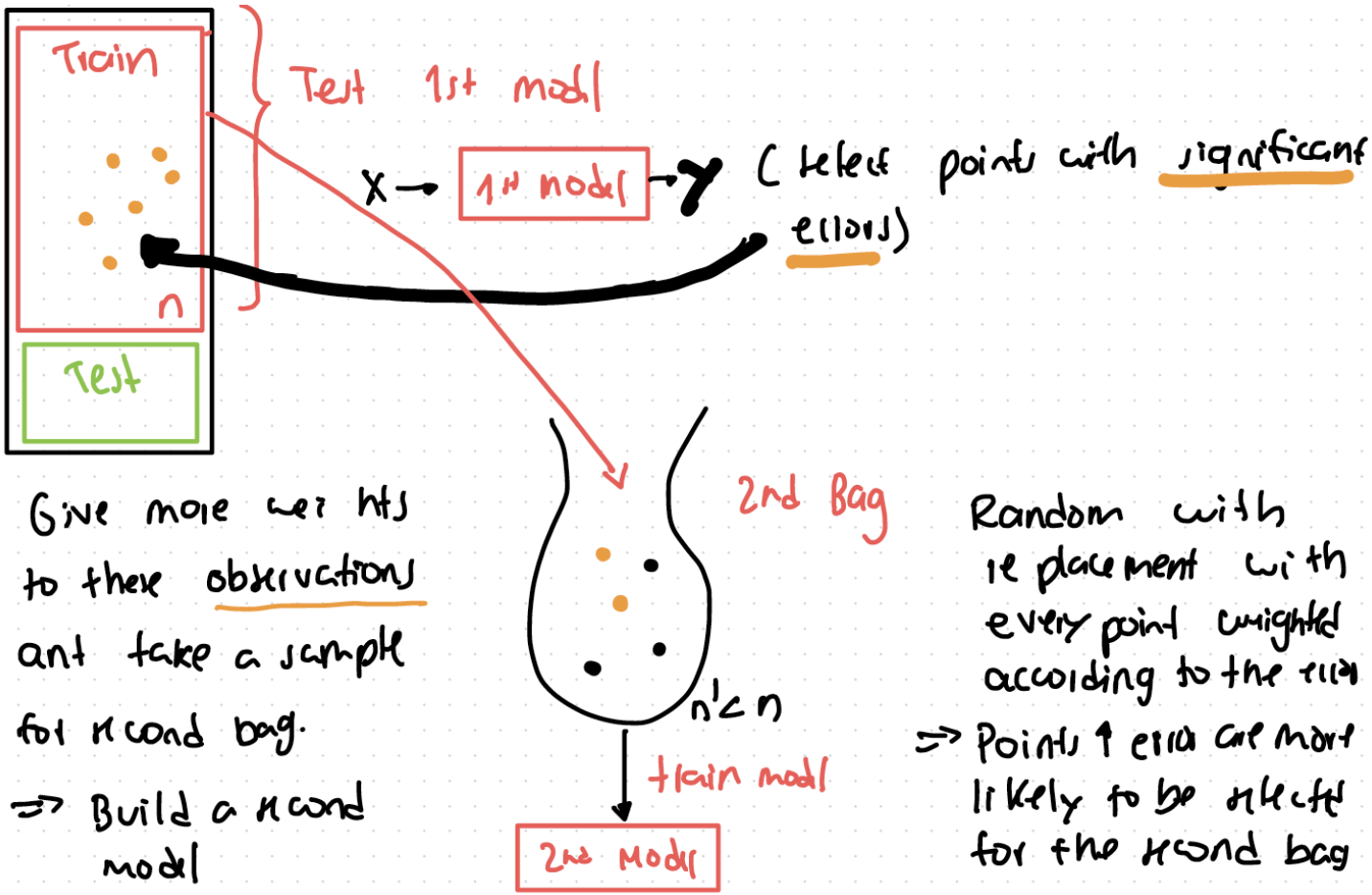
- **Number of trees** should not be too large, because unlike bagging and random forest, boosting can overfit.
- **Shrinkage parameter, or learning rate** controls speed of learning. Typical values are 0.1 or 0.01.
- **Interaction depth** is the number of splits in each tree. One split per tree often works well.
- **Subsampling** uses only part of the data (typically $\frac{1}{2}$ or less) to improve predictive performance.

Use cross-validation to find promising values.

Boosting idea 1



Boosting idea 2



Boosting idea 3

4) Test system (Model 1 + Model 2) all together

find again the observations that are not well predicted

5) Repeat ③-④ until the last B-th Bag ...

Pros and cons of boosted trees

Pros

- very high predictive performance if tuning parameters are chosen wisely
- each iteration improves the solution
- easily incorporates missing values
- can deal with larger data sets than random forest

Cons

- harder to tune because of larger number of parameters
- considerable chance of overfitting
- relatively long training time
- lack of interpretability (possible solution: variable importance measures)

Summary

- Decision trees are simple and interpretable models for regression and classification.
- However they are often not competitive with other methods in terms of prediction accuracy.
- Bagging, random forests (and boosting) are good methods for improving the prediction accuracy of trees.
- They work by growing many trees on the training data and then combining the predictions of the resulting ensemble of trees.
- Random forests and boosting are among the state-of-the-art methods for supervised learning - however their results can be difficult to interpret.